

第40章 全彩 LED 灯实验

本章讲解的实验工程目录为：固件库例程\TIM—全彩 LED 灯。

学习本章需要 TIM 定时器章节为基础。

40.1 全彩 LED 灯简介

全彩 LED 灯，实质上是一种把红、绿、蓝单色发光体集成到小面积区域中的 LED 灯，控制时对这三种颜色的灯管输出不同的光照强度，即可混合得到不同的颜色，其混色原理与光的三原色混合原理一致。

例如，若红绿蓝灯都能控制输出光照强度为[0:255]种等级，那么该灯可混合得到使用 RGB888 表示的所有颜色(包括以 RGB 三个灯管都全灭所表示的纯黑色)。

40.2 全彩 LED 灯控制原理

本教程配套开发板中的 RGB 灯就是一种全彩 LED 灯，前面介绍 LED 基本控制原理的时候，只能控制 RGB 三色灯的亮灭，即 RGB 每盏灯有[0:1]两种等级，因此只能组合出 8 种颜色。

要使用 STM32 控制 LED 灯输出多种亮度等级，可以通过控制输出脉冲的占空比来实现，见图 40-1。

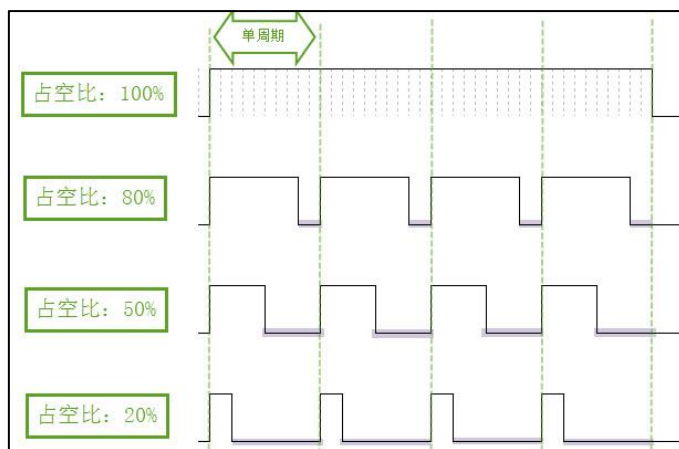


图 40-1 不同占空比的 PWM

示例图中列出了周期相同而占空比分别为 100%、80%、50 和 20%的脉冲波形，假如利用这样的脉冲控制 LED 灯，即可控制 LED 灯亮灭时间长度的比例。若提高脉冲的频率，LED 灯将会高频率进行开关切换，由于视觉暂留效应，人眼看不到 LED 灯的开关导致的闪烁现象，而是感觉到使用不同占空比的脉冲控制 LED 灯时的亮度差别。即单个控制周期内，LED 灯亮的平均时间越长，亮度就越高，反之越暗。

把脉冲信号占空比分成 256 个等级，即可用于控制 LED 灯输出 256 种亮度，使用三种这样的信号控制 RGB 灯即可得到 256*256*256 种颜色混合的效果。而要控制占空比，直接使用 STM32 定时器的 PWM 功能即可。

40.3 硬件设计

本小节讲解的实验工程目录为：固件库例程\TIM—全彩 LED 灯。

本实验中使用的 RGB 灯硬件与前面 LED 灯章节中的完全相同，见图 40-2，这是一个 RGB 灯，里面由红蓝绿三个小灯构成，使用 PWM 控制时可以混合成 256 不同的颜色。本实验与 LED 灯基础章节的主要差异是软件中的控制方式。

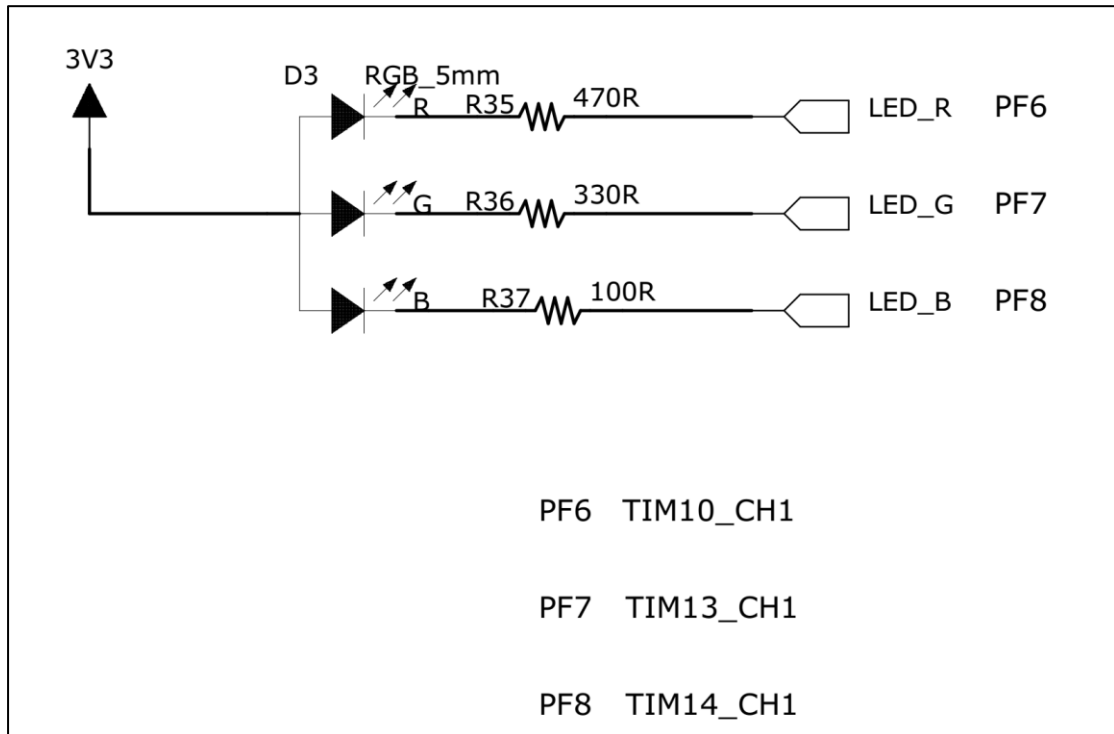


图 40-2 LED 硬件原理图

本开发板中设计的 RGB 灯控制引脚是经过仔细选择的，因为本实验的软件将使用 STM32 的定时器控制 PWM 脉冲的占空比，然而并不是任意 GPIO 都具有 STM32 定时器的输出通道功能，所以在设计硬件时，需要根据《STM32 数据手册》中的说明，选择具有定时器输出通道功能的引脚来控制 RGB 灯，见图 40-3。

Pin number						Pin name (function after reset) ⁽¹⁾	Pin type	I/O structure	Notes	Alternate functions	Additional functions
LQFP64	WLCSP90	LQFP100	LQFP144	UFBGA176	LQFP176						
-	-	-	18	K2	24	PF6	I/O	FT	(4)	TIM10_CH1 / FSMC_NIORD/ EVENTOUT	ADC3_IN4
-	-	-	19	K1	25	PF7	I/O	FT	(4)	TIM11_CH1/FSMC_NREG/ EVENTOUT	ADC3_IN5
-	-	-	20	L3	26	PF8	I/O	FT	(4)	TIM13_CH1 / FSMC_NIOWR/ EVENTOUT	ADC3_IN6

图 40-3 《STM32 数据手册》中本设计中使用的 LED 引脚说明

本实验中的 RGB 灯使用阴极分别连接到了 PF6、PF7 及 PF8，它们分别是定时器 TIM10、11、13 的通道 1。在本硬件设计方案中三个引脚使用了不同的定时器，它会给设计程序时会带来不便，若是在硬件资源分配充足的情况下，建议这三个控制引脚使用同一个定时器的不同通道。

40.4 软件设计

本工程中关于 RGB 灯控制的代码都存储在“bsp_color_led.c”及“bsp_color_led.h”文件，这些文件也可根据您的喜好命名，它们不属于 STM32 标准库的内容，是由我们自己根据应用需要编写的。

40.4.1 编程要点

- (1) 初始化 RGB 灯使用的 GPIO；
- (2) 配置定时器输出 PWM 脉冲；
- (3) 编写修改 PWM 脉冲占空比大小的函数；
- (4) 测试配置的定时器脉冲控制周期是否会导致 LED 灯明显闪烁；

40.4.2 代码分析

1. LED 灯硬件相关宏定义

为方便迁移代码适应其它硬件设计，实验中把硬件相关的部分使用宏定义到 bsp_color_led.h 文件中，使用不同硬件设计时，修改该文件即可，见代码清单 40-1。

代码清单 40-1 硬件相关宏定义（bsp_color_led.h 文件）

```
1 /*
2 !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
3 本工程的 LED 灯各通道使用的不是同一个定时器
4 设计硬件时建议使用同一个定时器来控制三盏灯，简化代码
5 若使用的硬件是同一个定时器，需要调整源代码和头文件，不能直接修改宏来实现
6 特别是定时器初始化部分。
7 !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
8 */
9 /*****定时器通道*****/
10 #define COLOR_TIM_GPIO_CLK (RCC_AHB1Periph_GPIOF)
11
12 /*****红灯*****/
13 #define COLOR_RED_TIM TIM10
14 #define COLOR_RED_TIM_CLK RCC_APB2Periph_TIM10
15 #define COLOR_RED_TIM_APBxClock_FUN RCC_APB2PeriphClockCmd
16 /*计算说明见 c 文件*/
17 /*部分通用定时器的时钟为 HCLK/4，部分为 HCLK/2，注意要把三个通道的定时器频率配置为一致*/
18 #define COLOR_RED_TIM_PRESCALER (((SystemCoreClock)/1000000)*30-1)
19
20 /*****绿灯*****/
21 #define COLOR_GREEN_TIM TIM11
```

```
22 #define COLOR_GREEN_TIM_CLK                RCC_APB2Periph_TIM11
23 #define COLOR_GREEN_TIM_APBxClock_FUN      RCC_APB2PeriphClockCmd
24 /*部分通用定时器的时钟为 HCLK/4，部分为 HCLK/2，注意要把三个通道的定时器频率配置为一致*/
25 #define COLOR_GREEN_TIM_PRESCALER          ((SystemCoreClock)/1000000)*30-1)
26
27 /*****蓝灯*****/
28 #define COLOR_BLUE_TIM                      TIM13
29 #define COLOR_BLUE_TIM_CLK                  RCC_APB1Periph_TIM13
30 #define COLOR_BLUE_TIM_APBxClock_FUN      RCC_APB1PeriphClockCmd
31 /*部分通用定时器的时钟为 HCLK/4，部分为 HCLK/2，注意要把三个通道的定时器频率配置为一致*/
32 #define COLOR_BLUE_TIM_PRESCALER          ((SystemCoreClock/2)/1000000)*30-1)
33
34
35
36 /*****红灯*****/
37
38 #define COLOR_RED_PIN                        GPIO_Pin_6
39 #define COLOR_RED_GPIO_PORT                 GPIOF
40 #define COLOR_RED_PINSOURCE                 GPIO_PinSource6
41 #define COLOR_RED_AF                        GPIO_AF_TIM10
42
43 #define COLOR_RED_TIM_OCxInit                TIM_OC1Init                //通道初始化函数
44 #define COLOR_RED_TIM_OCxPreloadConfig      TIM_OC1PreloadConfig //通道重载配置函数
45
46 //通道比较寄存器，以 TIMx->CCRx 方式可访问该寄存器，设置新的比较值，控制占空比
47 //以宏封装后，使用这种形式：COLOR_TIMx->COLOR_RED_CCRx，可访问该通道的比较寄存器
48 #define COLOR_RED_CCRx                      CCR1
49
50 /*****绿灯*****/
51 #define COLOR_GREEN_PIN                      GPIO_Pin_7
52 #define COLOR_GREEN_GPIO_PORT                 GPIOF
53 #define COLOR_GREEN_PINSOURCE                 GPIO_PinSource7
54 #define COLOR_GREEN_AF                        GPIO_AF_TIM11
55
56 #define COLOR_GREEN_TIM_OCxInit                TIM_OC1Init                //通道初始化函数
57 #define COLOR_GREEN_TIM_OCxPreloadConfig      TIM_OC1PreloadConfig //通道重载配置函数
58
59 //通道比较寄存器，以 TIMx->CCRx 方式可访问该寄存器，设置新的比较值，控制占空比
60 //以宏封装后，使用这种形式：COLOR_TIMx->COLOR_GREEN_CCRx，可访问该通道的比较寄存器
61 #define COLOR_GREEN_CCRx                      CCR1
62
63 /*****蓝灯*****/
64 #define COLOR_BLUE_PIN                      GPIO_Pin_8
65 #define COLOR_BLUE_GPIO_PORT                 GPIOF
66 #define COLOR_BLUE_PINSOURCE                 GPIO_PinSource8
67 #define COLOR_BLUE_AF                        GPIO_AF_TIM13
68
69 #define COLOR_BLUE_TIM_OCxInit                TIM_OC1Init                //通道初始化函数
70 #define COLOR_BLUE_TIM_OCxPreloadConfig      TIM_OC1PreloadConfig //通道重载配置函数
71
72 //通道比较寄存器，以 TIMx->CCRx 方式可访问该寄存器，设置新的比较值，控制占空比
73 //以宏封装后，使用这种形式：COLOR_TIMx->COLOR_BLUE_CCRx，可访问该通道的比较寄存器
74 #define COLOR_BLUE_CCRx                      CCR1
75
```

这些宏定义非常丰富，包括使用的定时器编号 COLOR_xxx_TIMx 和定时器时钟使能库函数 COLOR_xxx_TIM_APBxClock_FUN。

控制各个 LED 灯时，每个颜色占用一个通道，这些与通道相关的也使用宏封装起来了，如：端口号 COLOR_xxx_TIM_LED_PORT、引脚号 COLOR_xxx_TIM_LED_PIN、通道初始化库函数 COLOR_xxx_TIM_OCxInit、通道重载配置库函数

COLOR_xxx_TIM_OCxPreloadConfig 以及通道对应的比较寄存器名 COLOR_xxx_CCRx。其中 xxx 宏中的 xxx 指 RED、GREEN 和 BLUE 三种颜色。

由于部分操作使用宏封装后不够直观，此处着重强调一下与通道相关的寄存器宏。为方便修改定时器某通道输出脉冲的占空比，初始化定时器后，可以直接使用形如“TIM10->CCR1=0xFFFFF”的代码修改定时器 TIM10 通道 1 的比较寄存器中的数值，使用本工程中的宏封装后，形式改为“COLOR_RED_TIMx ->COLOR_RED_CCRx=0xFFFFF”。

2. 初始化 GPIO

首先，初始化用于定时器输出通道的 GPIO，见代码清单 40-2。

代码清单 40-2 初始化 GPIO

```
1 /**
2  * @brief 配置 TIM 复用输出 PWM 时用到的 I/O
3  * @param 无
4  * @retval 无
5  */
6 static void TIMx_GPIO_Config(void)
7 {
8     /*定义一个 GPIO_InitTypeDef 类型的结构体*/
9     GPIO_InitTypeDef GPIO_InitStructure;
10
11     /*开启 LED 相关的 GPIO 外设时钟*/
12     RCC_AHB1PeriphClockCmd ( COLOR_TIM_GPIO_CLK, ENABLE);
13
14     GPIO_PinAFConfig(COLOR_RED_GPIO_PORT,COLOR_RED_PINSOURCE,COLOR_RED_AF);
15     GPIO_PinAFConfig(COLOR_GREEN_GPIO_PORT,COLOR_GREEN_PINSOURCE,COLOR_GREEN_AF);
16     GPIO_PinAFConfig(COLOR_BLUE_GPIO_PORT,COLOR_BLUE_PINSOURCE,COLOR_BLUE_AF);
17
18     /*COLOR_LED1*/
19     GPIO_InitStructure.GPIO_Pin = COLOR_RED_PIN;
20     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
21     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
22     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
23     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
24
25     GPIO_Init(COLOR_RED_GPIO_PORT, &GPIO_InitStructure);
26
27     /*COLOR_LED2*/
28     GPIO_InitStructure.GPIO_Pin = COLOR_GREEN_PIN;
29     GPIO_Init(COLOR_GREEN_GPIO_PORT, &GPIO_InitStructure);
30
31     /*COLOR_LED3*/
32     GPIO_InitStructure.GPIO_Pin = COLOR_BLUE_PIN;
33     GPIO_Init(COLOR_BLUE_GPIO_PORT, &GPIO_InitStructure);
34
35 }
```

与 LED 灯的基本控制不同，由于本实验直接使用定时器输出通道的脉冲信号控制 LED 灯，此处代码把 GPIO 相关的引脚都配置成了复用推挽输出模式，后面将使用定时器控制引脚进行 PWM 输出。

3. 定时器 PWM 配置

接着配置定时器输出 PWM 的工作模式，见代码清单 40-3。

代码清单 40-3 定时器 PWM 配置

```
1  /**
2   * @brief 配置 COLOR_TIMx 输出的 PWM 信号的模式，如周期、极性
3   * @param 无
4   * @retval 无
5   */
6  static void TIM_Mode_Config(void)
7  {
8      TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
9      TIM_OCInitTypeDef  TIM_OCInitStructure;
10
11      COLOR_RED_TIM_APBxClock_FUN(COLOR_RED_TIM_CLK,ENABLE);
12      COLOR_GREEN_TIM_APBxClock_FUN(COLOR_GREEN_TIM_CLK,ENABLE);
13      COLOR_BLUE_TIM_APBxClock_FUN(COLOR_BLUE_TIM_CLK,ENABLE);
14
15
16      /*注意要把三个通道的定时器频率配置为一致*/
17
18      /* 累计 TIM_Period 个后产生一个更新或者中断*/
19      //当定时器从 0 计数到 255，即为 256 次，为一个定时周期
20      TIM_TimeBaseStructure.TIM_Period = 256-1;
21
22      // APB1 定时器时钟源 TIMxCLK = 2 * PCLK1
23      //      PCLK1 = HCLK / 4
24      //      => TIMxCLK = HCLK / 2 = SystemCoreClock / 2
25      // 定时器频率为 = TIMxCLK/(TIM_Prescaler+1) =
26      //      (SystemCoreClock / 2)/((SystemCoreClock/2)/1000000)*30 =
27      //      1000000/30 = 1/30MHz
28
29      //APB2 定时器时钟源 TIMxCLK = 2 * PCLK2
30      //      PCLK2 = HCLK / 2
31      //      => TIMxCLK = HCLK = SystemCoreClock
32      // 定时器频率为 = TIMxCLK/(TIM_Prescaler+1) =
33      //      (SystemCoreClock )/((SystemCoreClock)/1000000)*30 = 1000000/30 =
34      //      1/30MHz
35
36      /*基本定时器配置 TIM_Prescaler 根据效果来设置，中断周期小 灯闪烁快，大则闪烁缓慢*/
37
38      TIM_TimeBaseStructure.TIM_Prescaler = COLOR_RED_TIM_PRESCALER;
39      //设置时钟分频系数：不分频(这里用不到)
40      TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1 ;
41      TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //向上计数模式
42
43      // 初始化定时器 TIMx
44      TIM_TimeBaseInit(COLOR_RED_TIM, &TIM_TimeBaseStructure);
45
46      /*基本定时器配置 TIM_Prescaler 根据效果来设置即可，中断周期小
47      灯闪烁快，大则闪烁缓慢*/
48      TIM_TimeBaseStructure.TIM_Prescaler = COLOR_GREEN_TIM_PRESCALER;
49      // 初始化定时器 TIMx
50      TIM_TimeBaseInit(COLOR_GREEN_TIM, &TIM_TimeBaseStructure);
51
52      /*基本定时器配置 TIM_Prescaler 根据效果来设置即可，中断周期小
53      灯闪烁快，大则闪烁缓慢*/
54      TIM_TimeBaseStructure.TIM_Prescaler = COLOR_BLUE_TIM_PRESCALER;
55      // 初始化定时器 TIMx
56      TIM_TimeBaseInit(COLOR_BLUE_TIM, &TIM_TimeBaseStructure);
57
58      /*PWM 模式配置*/
59      /* PWM1 Mode configuration: Channel1 */
```



```
60 TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1; //配置为 PWM 模式 1
61 TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable; //使能输出
62 TIM_OCInitStructure.TIM_Pulse = 0; //设置初始 PWM 脉冲宽度为 0
63 //当定时器计数值小于 CCR_Val 时为低电平 LED 灯亮
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_Low;
64
65
66 //使能通道
67 COLOR_RED_TIM_OCxInit(COLOR_RED_TIM, &TIM_OCInitStructure);
68 /*使能通道重载*/
69 COLOR_RED_TIM_OCxPreloadConfig(COLOR_RED_TIM, TIM_OCPreload_Enable);
70
71 //使能通道
72 COLOR_GREEN_TIM_OCxInit(COLOR_GREEN_TIM, &TIM_OCInitStructure);
73 /*使能通道重载*/
74 COLOR_GREEN_TIM_OCxPreloadConfig(COLOR_GREEN_TIM, TIM_OCPreload_Enable);
75
76 //使能通道
77 COLOR_BLUE_TIM_OCxInit(COLOR_BLUE_TIM, &TIM_OCInitStructure);
78 /*使能通道重载*/
79 COLOR_BLUE_TIM_OCxPreloadConfig(COLOR_BLUE_TIM, TIM_OCPreload_Enable);
80
81 //使能 TIM 重载寄存器 ARR
82 TIM_ARRPreloadConfig(COLOR_RED_TIM, ENABLE);
83 TIM_ARRPreloadConfig(COLOR_GREEN_TIM, ENABLE);
84 TIM_ARRPreloadConfig(COLOR_BLUE_TIM, ENABLE);
85
86
87 // 使能计数器
88 TIM_Cmd(COLOR_RED_TIM, ENABLE);
89 TIM_Cmd(COLOR_GREEN_TIM, ENABLE);
90 TIM_Cmd(COLOR_BLUE_TIM, ENABLE);
91
92 }
```

定时器配置的主体跟前面《PWM 互补输出实验》小节介绍的实验类似，关键代码已用红色字体加粗显示。

本配置初始化了控制 RGB 灯用的定时器，它被配置为向上计数，TIM_Period 被配置为 255，即定时器每个时钟周期计数器加 1，计数至 255 时溢出，从 0 开始重新计数；而代码中的 PWM 通道配置，当计数器的值小于输出比较寄存器 CCRx 的值时，PWM 通道输出低电平，点亮 LED 灯。所以上述代码配置把输出脉冲的单个周期分成了 256 份（注意区分定时器的时钟周期和输出脉冲周期），而输出比较寄存器 CCRx 配置的值即该脉冲周期内 LED 灯点亮的时间份数，所以修改 CCRx 的值，即可控制输出[0:255]种亮度等级。

关于定时器中的 TIM_Prescaler 分频配置，只要让它设置使得 PWM 控制脉冲的频率足够高，让人看不出 LED 灯闪烁即可，您可以亲自修改使用其它参数测试。

4. 颜色混合

初始化完定时器和 PWM 输出通道后，再编写用于设置混合颜色的函数，见代码清单 40-4。

代码清单 40-4 颜色混合

```
1 #define COLOR_RED_TIM TIM10
2 #define COLOR_GREEN_TIM TIM11
3 #define COLOR_BLUE_TIM TIM13
4
```

```
5 #define COLOR_RED_CCRx          CCR1
6 #define COLOR_GREEN_CCRx       CCR1
7 #define COLOR_BLUE_CCRx        CCR1
8
9 /**
10  * @brief 设置 RGB LED 的颜色
11  * @param rgb:要设置 LED 显示的颜色值格式 RGB888
12  * @retval 无
13  */
14 void SetRGBColor(uint32_t rgb)
15 {
16     //根据颜色值修改定时器的比较寄存器值
17     COLOR_RED_TIM->COLOR_RED_CCRx = (uint8_t)(rgb>>16); //R
18     COLOR_GREEN_TIM->COLOR_GREEN_CCRx = (uint8_t)(rgb>>8); //G
19     COLOR_BLUE_TIM->COLOR_BLUE_CCRx = (uint8_t)rgb; //B
20 }
21
22 /**
23  * @brief 设置 RGB LED 的颜色
24  * @param r\g\b:要设置 LED 显示的颜色值
25  * @retval 无
26  */
27 void SetColorValue(uint8_t r,uint8_t g,uint8_t b)
28 {
29     //根据颜色值修改定时器的比较寄存器值
30     COLOR_RED_TIM->COLOR_RED_CCRx = r;
31     COLOR_GREEN_TIM->COLOR_GREEN_CCRx = g;
32     COLOR_BLUE_TIM->COLOR_BLUE_CCRx = b;
33 }
34
```

本工程提供 SetRGBColor 和 SetColorValue 这两个函数用于设置 RGB 灯的颜色值，这两个函数功能和原理都一样，只是输入参数格式不同，方便使用不同格式的颜色值。

在代码层面，控制 RGB 灯的颜色实质就是控制各个 PWM 通道输出脉冲的占空比，而占空比可以通过设置定时器相应通道的输出比较寄存器值修改，又因为定时器已经把单个控制脉冲周期分成[0:255]份，控制时只要把 RGB888 各通道的颜色值直接赋予给输出比较寄存器即可。

5. 主函数

代码清单 40-5 主函数

```
1 /**
2  * @brief 主函数
3  * @param 无
4  * @retval 无
5  */
6 int main(void)
7 {
8     uint32_t random_color = 0;
9
10     /* 初始化呼吸灯 */
11     ColorLED_Config();
12
13     /*初始化串口*/
14     Debug_USART_Config();
15
16     /* 使能 RNG 时钟 */
17     RCC_AHB2PeriphClockCmd(RCC_AHB2Periph_RNG, ENABLE);
18     /* 使能 RNG 外设 */

```



```
19     RNG_Cmd(ENABLE);
20
21     printf("\r\n 欢迎使用野火  STM32 F407 开发板。 \r\n");
22     printf("\r\n 全彩 LED 灯例程 \r\n");
23     printf("\r\n 使用 PWM 控制 RGB 灯，可控制输出各种颜色 \r\n ");
24
25     while (1) {
26         SetRGBColor(COLOR_YELLOW);
27         Delay(0xFFFFFFFF);
28
29         /* 等待随机数产生完毕 */
30         while (RNG_GetFlagStatus(RNG_FLAG_DRDY) == RESET);
31         /* 获取随机数 */
32         random_color = RNG_GetRandomNumber();
33
34         printf("\r\n 随机颜色值: 0x%06x", random_color & 0xFFFFFFFF);
35         /* 显示随机颜色 */
36         SetRGBColor(random_color & 0xFFFFFFFF);
37         Delay(0x2FFFFFFF);
38
39     }
40 }
```

main 函数中直接调用了 ColorLED_Config 函数，而该函数内部又直接调用了前面讲解的 GPIO 和 PWM 配置函数：TIMx_GPIO_Config 和 TIM_Mode_Config。初始化完定时器后，又初始化了 STM32F4 芯片的随机数发生器外设 RNG，使用它产生随机数，然后调用 SetRGBColor 和 SetColorValue 把 RGB 灯切换成该随机颜色。实际应用中也可以根据自己的实际需要向函数设置 RGB565 的颜色值。

40.4.3 下载验证

编译并下载本程序到开发板，给开发板上电复位，可看到 RGB 彩灯显示不同的色彩。